

Technological benchmark

Overdrive

Aventix
Epitech Paris
February 13, 2026

Summary

OverDrive – Technological benchmark

1- OverDrive Project Context

2- Objectives of the technological benchmark

3- Reminder of target uses for the test phase

4- Benchmark of the VR brick

- 1- Common test scenario (VR)
- 2- Comparison of VR solutions
- 3- Qualitative reading (premium / feasibility)

5. Backend - benchmark Node/TypeScript vs Go

- 1- Reference backend scope
- 2- Common test scenario (backend)
- 3- Backend comparison

6- Mobile - technology benchmark

- 1- Mobile perimeter (companion)
- 2- Common test scenario (mobile)
- 3- Mobile comparison

7- Database - benchmark

- 1- DB perimeter
- 2- Comparative DB / ORM

8. Conclusion - Synthesis and Realistic Choices

1- OverDrive Project Context

OverDrive is a student project aimed at designing an **immersive platform** around motorsport, combining **virtual reality** and **real-time data visualization**.

The main challenge is to explore a qualitative alternative to classic 2D interfaces: **present race information (rankings, deviations, tires, weather, key events)** in a structured, readable and modular VR environment.

At this stage, the goal is not to immediately deliver a complete product on a large scale, but to :

- **To define a realistic architecture**, consistent with a gradual increase in load,
- to test different key technologies (VR engine, real-time backend, companion mobile app),
- to select a viable and sustainable stack for the rest of the project.

2- Objectives of current work

The current phase aims at a targeted **technological benchmark**, mainly on the VR brick, taking into account:

- **Team constraints:** 5 developers, 1 to 1.5 years of experience, reasonable learning capacity.
- **Hardware constraints:** Meta Quest 3/3S target platform, heterogeneous PC machines.
- **Minimum quality requirements:** smooth application performance, low latency, premium readability, attention to accessibility.

Specifically, this benchmark should make it possible to :

- Identify 2 to 3 realistic VR solutions (Unity, Unreal, WebXR).
- Implement **comparable prototypes**, representative of OverDrive.
- Measure, for each solution: performance, ease of development, backend integration, and ability to evolve.

The objective is not to cover all the final use cases, but to **validate a technical foundation** capable of supporting a gradual evolution.

3- Minimal reminder of target uses

As part of the benchmark, the uses are deliberately limited and oriented "technical validation" :

- Display in VR of **essential data** (ranking, deviations, events).
- Simple distinction between:
 - A "**simplified**" **mode** (non-expert profile),
 - A "**detailed**" **mode** (expert profile), Via the density and prioritization of information.
- **Light customization** of panels (position / size) + backup via backend.

Ce périmètre suffit à comparer efficacement des moteurs VR, sans surcharger le prototype de storytelling ou de scénarios complexes

4- VR brick – perimeter and benchmark

1- Common test scenario (VR)

All solutions are evaluated on a strictly identical scenario :

- **Helmet** : Meta Quest 3 / 3S
- **Backend** : API REST + WebSocket (Test backend or mock)
- VR scene :
 - An OverDrive room (minimalist environment),
 - a **mini 3D circuit** (low poly) + **small cars** updated almost in real time,
 - 2 to 4 data panels (rankings, gaps, events),
 - 1 video panel (replay / live depending on availability).

2- Comparison of VR solutions

Solution	What we test concretely	Expected strengths	Limits / probable risks
Unity (native Quest)	Quest scene + 3D circuit + cars + 3D UI + WebSocket + video	Excellent perf/simplicity compromise on Quest, huge ecosystem, fast iterations, very "VR product-ready"	Requires an optimization discipline (3D + UI + video), network traps (TLS, WS Android) to anticipate

Solution	What we test concretely	Expected strengths	Limits / probable risks
Unreal Engine	Equivalent scene, same data + video streams (less dense prototype)	Very good rendering possible, powerful optimization tools	Heavy learning curve, often oversized for a data-first app, risk of timebox exploded
WebXR (browser Quest)	WebXR experience: "light" circuit + 3D panels + video + WebSocket	Ultra-fast deployment (URL), homogeneous web stack, perfect for demo	Browser-dependent performance/UX, variable stability, video + 3D + UI can quickly limit

3- Qualitative reading (premium / feasibility)

- **Unity** is the most consistent candidate if the objective is a **premium main VR** on Quest, with a good balance between rendering, performance and iteration speed.
- **Unreal** is relevant if the priority objective becomes the "kinematic" rendering, but presents a strong risk in a student context (time + complexity).
- **WebXR** is very interesting for a **demo/alternative** web version, but more risky as a main VR platform.

5- Backend - Node/TypeScript vs Go benchmark

1- Reference backend perimeter

The OverDrive backend exposes :

- A **REST API** : accounts, presets/layouts, event catalog and metadata,
- A **WebSocket** : real-time broadcast (positions, ranking, simple events).

The backend remains agnostic of customers: VR, mobile and web consume the **same contract**.

2- Common test scenario (backend)

API REST

- Simplified Auth: login/register
- /Events, /events/:id/sessions

- /Presets, /presets/:id

WebSocket

- Session flow (state, flags, ranking): **realtime:session:<id>:state**
- Car position flow : **realtime:session:<id>:driver:<driverId>** Or aggregate stream

Simulated load

- 5–20 updates/s per session
- 1–5 updates/s per tracked pilot
- a few simultaneous clients (VR + mobile + web)

3- Comparative backend

Solution	What we test	Expected strengths	Limits / probable risks
Go	REST + WebSocket + PostgreSQL + Redis	Very good real time, low CPU consumption, simplicity of the “ingest → broadcast” model, excellent for frequent streams	Less “monolanguage” synergy if front in TS, more boilerplate according to frameworks
Node/TypeScript	REST + WS (ws/Socket.IO) + PostgreSQL + Redis	Massive ecosystem, monolanguage with RN/web, strong DX (OpenAPI, validation, auth)	Lower gross perf at equal load on intensive WS, requires async discipline
Mix (optional)	Go for real time + Node for services	Combine perf Go + Node comfort	Multi-service complexity, probably useless in student projects

Recommended reading

- If the priority ambition is **"clean" and robust real time**: Go is **very consistent**.
- If the main criterion is full TS (web/mobile) alignment: **Node/TS is very viable** in a student setting (limited load).

6- Mobile – technology benchmark

1- Mobile perimeter (companion)

The mobile is not the main showcase brick, but a premium component of the ecosystem :

- VR parallel tracking,
- notifications,
- simplified mode / detailed mode,
- access to highlights.

2- Common test scenario (mobile)

- iOS + Android
- REST + WebSocket (same endpoints as VR)
- 1 live screen: simplified ranking, gaps, key events
- simplified toggles - detailed
- accessibility requirements: contrast, text sizes, information density

3- Mobile comparative

Solution	What we test	Expected strengths	Limits / probable risks
Flutter	iOS/Android app + WebSocket + themes + accessibility	Very good cross-platform, high-performance rendering, fine UI control, fast if existing base	Less aligned if web front in TS, pay attention to refresh (battery/perf)
React Native	Equivalent app (REST/WS + UI live)	JS/TS monolanguage with Node and web backend, huge ecosystem	Sometimes painful native bridging, perf to watch, learning cost if team already Flutter
Swift/Kotlin natif	Minimal live screen by OS	Excellent perfs, total OS control	Doubled, unrealistic time on multi-bricke project in 1–1.5 years

Recommendation (coherent student project + premium)

- **Flutter** is the leading candidate (time-to-market + UI quality + existing user base).
- Native Redundancy and native Redundancy are theoretical alternatives to keep in mind, without in-depth exploration at this stage.

7- Database – benchmark

1- DB perimeter

- Persistant : utilisateurs, sessions d'authentification, fournisseurs, événements/sessions, pilotes/équipes, pré-réglages/mises en page
- Temps réel : Redis (état live), (option) base time-series pour historique télémétrie

2- DB / ORM Comparison

Solution	What we test	Expected strengths	Limits / probable risks
PostgreSQL + Prisma	schéma + migrations + CRUD presets/users	Typage fort, migrations propres, DX excellent en TS	Not suitable for a Go backend
PostgreSQL + ORM Go (GORM/sqlc)	Same diagram + catalog queries/presets	Aligned Go, solid perf, fine control	DX less “batteries included” than Prisma
PostgreSQL + SQL “light”	Manual requests	Total control, SQL pedagogy, predictable perf	More verbal, risk of errors, heavier maintenance

Simple recommendation (according to backend)

- Backend **Go** → PostgreSQL + **GORM or sqlc** + Redis (+ TimescaleDB option later).
- Backend **Node/TS** → PostgreSQL + **Prisma** + Redis (+ option time-series).

Conclusion – Synthesis and realistic choices

The benchmark aims to select a **realistic** stack, controllable by the team, and able to support a **premium** rendering without overinvesting in complexity. Recommended orientation (consistent with constraints and target quality)

- Main VR: Unity (Quest native)
- Real-time backend: Go or Node/TS (decision guided by the team organization, not by the VR client)
- Mobile companion: Flutter
- Data: PostgreSQL + Redis (+ optional time-series)

This selection makes it possible to prototype quickly, ensure good rendering quality on Quest, and maintain a scalable architecture towards more ambitious features (advanced telemetry, multi-sessions, replays, etc.).